

Constraint Boundary Wandering Framework: Enhancing Constrained Optimization With Deep Neural Networks

Shuang Wu, Shixiang Chen¹, Li Shen², Lefei Zhang³, *Senior Member, IEEE*, and Dacheng Tao⁴, *Fellow, IEEE*

Abstract—Constrained optimization problems are pervasive in various fields, and while conventional techniques offer solutions, they often struggle with scalability. Leveraging the power of deep neural networks (DNNs) in optimization, we present a novel learning-based approach, the Constraint Boundary Wandering Framework (CBWF), to address these challenges. Our contributions include introducing a boundary wandering strategy inspired by the active-set method, enhancing equality constraint feasibility, and treating the Lipschitz constant as a learnable parameter. Additionally, we evaluate the regularization term, illustrating that the nonsmooth L2 norm yields superior results. Extensive testing on synthetic datasets and the ACOPT dataset demonstrates CBWF's superiority, outperforming existing deep learning-based solvers in terms of both objective and constraint loss.

Index Terms—Learning-based optimization, constrained optimization, power system.

I. INTRODUCTION

CONSTRAINED optimization plays a pivotal role in diverse domains, including power systems, weather and climate modeling, materials science, and beyond [1]. While conventional optimization techniques have proven effective in

addressing constrained optimization problems, they often incur significant computational costs, especially when dealing with large-scale problems. As a result, there has been a growing interest in leveraging the impressive capabilities of deep neural networks (DNNs) across various applications to tackle such challenges. Despite the successes of DNNs in diverse fields, the application of learning-based methods to solve optimization problems with constraints remains relatively limited. The DC3 algorithm [2] introduces a novel learning-based approach to solve a wide range of equality and inequality constrained optimization problems, representing a pioneering effort in this area and showing the efficiency of using a learning-based method. To solve the optimization problem, DC3 trains a deep neural network using the data as input and minimizes the penalized objective function, which includes a regularization term to enforce the constraints and the original objective function. During training process, DC3 dissolves the equality constraints via a variable elimination process and reformulates inequality constrained problems using a penalty method, which involves adding a regularization term to the objective function to enforce the constraints. Specifically, the equality constraint can be satisfied using an equality completion process, which fills in missing information to ensure the constraint to be satisfied, while the inequality constraint is enforced through a correction process, which adjusts the solution to satisfy the constraint. During the testing phase, DC3 can efficiently solve a new problem by performing a forward pass of the trained neural network to compute the solution, including a correction step to ensure the solution satisfies the constraints. However, while DC3 proposes a general framework with an acceptable constraint loss, it still suffers from the loss of objective information during the correction process, resulting in relatively higher objective value to the optimization problem.

This paper introduces modifications to the DC3 algorithm, with a primary emphasis on achieving superior performance in handling inequality constraints and obtaining better results in terms of the optimization objective value. We integrate the active-set method [3], [4] into the DC3 framework. The central challenge we tackle is the loss of objective information, particularly during the correction step. To address this issue, we present a novel two-stage constrained optimization framework called the Constraint Boundary Wandering Framework (CBWF). In the first stage, we follow the DC3 methodology, making adjustments to the correction step and incorporating

Received 10 December 2023; revised 28 August 2024; accepted 28 March 2025. Date of publication 15 April 2025; date of current version 3 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62431020, in part by the Foundation for Innovative Research Groups of Hubei Province under Grant 2024AFA017, in part by Anhui Provincial Science and Technology Innovation Major Program under Grant 202423e09050003, in part by Shenzhen Basic Research Project (Natural Science Foundation) Basic Research Key Project under Grant JCYJ20241202124430041, and in part by National Research Foundation, Singapore, under its NRF Professorship under Grant NRF-P2024-001. Recommended for acceptance by S.K. Zhou. (Corresponding authors: Lefei Zhang; Shixiang Chen.)

Shuang Wu and Lefei Zhang are with the National Engineering Research Center for Multimedia Software, School of Computer Science, Wuhan University, Wuhan 430072, China (e-mail: wushatxs@whu.edu.cn; zhanglefei@whu.edu.cn).

Shixiang Chen is with the Key Laboratory of the Ministry of Education for Mathematical Foundations and Applications of Digital Technology, School of Mathematical Sciences, University of Science and Technology of China, Hefei 230026, China (e-mail: shxchen@ustc.edu.cn).

Li Shen is with the School of Cyber Science and Technology, Shenzhen Campus of Sun Yat-sen University, Shenzhen 518107, China (e-mail: mathshenli@gmail.com).

Dacheng Tao is with the College of Computing and Data Science, Nanyang Technological University, Singapore 639798 (e-mail: dacheng.tao@gmail.com).

This article has supplementary downloadable material available at <https://doi.org/10.1109/TPAMI.2025.3560762>, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2025.3560762

an updated regularization loss objective. In the second stage, inspired by the classical active-set method, our approach focuses on boundary exploration near inequality constraints, aiming to achieve a lower objective value. This improvement leads to a superior objective loss. For the boundary wandering strategy, different from the classical active-set method, we use a gradient based method to impose decrease of objective. Additionally, recognizing the computational costs associated with obtaining the gradient Lipschitz constant, we introduce a data-driven method for estimating this constant, which contributes to the improved performance of our algorithm.

The primary contributions of this work are summarized as follows.

- Drawing inspiration from active sets, we introduce a novel Boundary Wandering Step (BWS) aimed at further reducing the objective value in optimization problems. This strategy guides the network to explore near the boundary of inequality constraints, thereby improving solution quality. Additionally, the BWS method, as a novel optimization algorithm, has been proven to converge and comes with theoretical guarantees.
- We propose a modification to the correction module, improving the feasibility of nonlinear equality constraints, particularly on the ACOPT dataset. This enhancement extends the applicability of the algorithm to a broader range of constrained optimization problems.
- In line with the principles of learning-based methods, we treat the Lipschitz constant as a learnable parameter. This innovative approach eliminates the need for manual strategies to determine this constant, leading to improved optimization outcomes.
- We evaluate the regularization term in the soft loss and discover that the nonsmooth L2 norm outperforms both L1 norm and L2 squared norm (as used by DC3), contributing to better constraint handling and overall optimization.

II. RELATED WORK

Learning to optimize: The intersection of optimization and deep learning has spurred significant research activity. In this section, we review key contributions in this domain, aiming to advance the optimization method performance or even produce a novel approach [5], [6], [7]. Learning to optimize (L2O) has attracted attention in recent years. This approach is well-suited for handling complex, high-dimensional data due to the ability of deep neural networks to learn nonlinear functions. Through offline training, the L2O method learns the relevant parameters and achieves fast speed or better performance in testing. Leveraging the Long Short Term Memory (LSTM) [8], [9], [10] unroll the iteration trajectories and train to learn the optimal module parameters. Rather than using the LSTMs for imitating the optimization process, there are also some works to simulate the analytic optimization algorithms. To solve the LASSO formulation, [11] unfolds the classical iterative shrinkage thresholding algorithm (ISTA) with the threshold and the step parameters becoming learnable. To extend the method for equality constraint, [12] unrolls the alternating direction method of multipliers (ADMM) algorithm [13] and also takes advantage

of the data-driven threshold and the step. It should be noted that these methods cannot guarantee strict feasibility, since the original constrained optimization problem has been transformed.

Differentiable constrained optimization solver: In the realm of constrained optimization problems, a prevalent strategy involves the use of a constraint projection operator [14] applied to the output layer of the neural network. This approach aligns with solving projection-like optimization problems via Karush-Kuhn-Tucker (KKT) systems [15], [16], [17] integrated within deep neural networks. A notable drawback of this method is the high computational cost incurred during the forward and backward passes, often leading to inefficiencies. Addressing these computational challenges, recent advancements have been made. For instance, Wang et al. [18] introduced an efficient solver specifically tailored for projections on positive linear constraints. In a similar vein, Gao et al. [19], [20] utilized Recurrent Neural Networks (RNNs) to emulate traditional manifold optimization algorithms on Riemannian manifolds. However, when confronted with more complex or highly nonconvex constraints, some studies [21], [22] resort to approximating the gradient of the projection through perturbation techniques. While effective, these methods are hampered by the necessity of extensive sampling, thereby escalating the computational cost.

Implicit layer: Implicit differential models, such as the optimization layer [16], ordinary differential equations (ODEs) [23], [24], and deep equilibrium models (DEQ) [25], [26], are increasingly replacing traditional feed-forward networks and are widely used in vision tasks, language models, and diffusion models [27]. These implicit layers solve various optimization problems, including fixed-point problems and differential equations. However, a significant drawback of these methods is their high computational complexity. Some approaches, such as [28], attempt to unfold classical ADMM, but this can lead to minor constraint violations in the test dataset and incur high computational costs.

To address these limitations, our work introduces a novel and efficient solution. We utilize a vanilla neural network architecture enhanced with computationally efficient layers, training a deep block specifically designed for this purpose. This method not only effectively handles constraints but also significantly reduces computational overhead, offering a more practical and efficient alternative for constrained optimization in deep learning.

III. RETHINKING DC3 METHOD

In this section, we introduce some basic concepts of constrained optimization problems. We revisit the DC3 method in Section III-B and discuss its limitations in Section III-C. Additionally, we briefly introduce the classical active-set method, providing a foundational understanding of the techniques that will guide our exploration in the subsequent sections.

A. Preliminary

Notation: The Rectified Linear Unit (ReLU) function is denoted by $\text{ReLU}(x) = \max\{x, 0\}$. The L2 norm of a vector x is denoted by $\|x\|_2$, and the L1 norm is denoted by $\|x\|_1$. For

a vector-valued function $g : \mathbb{R}^m \rightarrow \mathbb{R}^l$, the Jacobian matrix is denoted by $Jg \in \mathbb{R}^{l \times m}$.

In this paper, we solve the following constrained optimization problem

$$\begin{aligned} \min_{y \in \mathbb{R}^n} \quad & f_x(y), \\ \text{s.t.} \quad & g_x(y) \leq 0, h_x(y) = 0, \end{aligned} \quad (1)$$

where $f_x(y)$ is a smooth function, $g_x(y)$ and $h_x(y)$ are also smooth. Formally, let $x \in \mathbb{R}^d$ denotes the problem data, and $y \in \mathbb{R}^n$ denotes the decision variable. We assume there are $n - m$ equality constraints and l inequality constraints. The equality and inequality constraints are denoted as

$$\begin{aligned} h_x(y) &= [h_1(y), h_2(y), \dots, h_{n-m}(y)]^\top = 0, \\ g_x(y) &= [g_1(y), g_2(y), \dots, g_l(y)]^\top \leq 0, \end{aligned}$$

where $h_i : \mathbb{R}^n \rightarrow \mathbb{R}$ and $g_j : \mathbb{R}^n \rightarrow \mathbb{R}$ for all $i = 1, \dots, n - m$ and $j = 1, \dots, l$. In our notation, $h_x : \mathbb{R}^n \rightarrow \mathbb{R}^{n-m}$ and $g_x : \mathbb{R}^n \rightarrow \mathbb{R}^l$. Given m entries of y , denoted by $z \in \mathbb{R}^m$, we assume, without loss of generality, that z corresponds to the first m entries of y . Accordingly, the Jacobian matrix Jh can be partitioned as $Jh = [Jh_1 \mid Jh_2]$, where $Jh_1 \in \mathbb{R}^{(n-m) \times m}$ consists of the first m th columns, and $Jh_2 \in \mathbb{R}^{(n-m) \times (n-m)}$ comprises the columns from $m + 1$ to n . If Jh_2 is invertible, the full entries of y can be expressed using the implicit function theorem. Specifically, there exists a *local* mapping $\varphi_x : U \rightarrow \mathbb{R}^{n-m}$, with $U \subset \mathbb{R}^m$, such that $0 = h_x(y) = h_x\left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix}\right)$. Moreover, the Jacobian matrix of φ_x in U is given by

$$\frac{\partial \varphi_x(z)}{\partial z} = -(Jh_2)^{-1} Jh_1. \quad (2)$$

Note that if the specific form of φ_x is unknown, one can approximate the function $h_x(y)$ by the linear form

$$l_{h_x}(y) := h_x(y) + Jh_x(y)(y_1 - y) \approx h_x(y_1),$$

where y_1 is close to y . Then, we locally treat the nonlinear constraint as the linear constraint $l_{h_x}(y) = 0$.

B. Revisit of DC3

Now, let us briefly revisit the DC3 method. DC3 is developed for training a deep neural network, denoted as $N_\Theta(x)$, with the aim to predict the solution y based on input x using the loss function $f_x(y)$, where Θ represents the network parameters. A salient challenge addressed by DC3 is the concurrent presence of equality and inequality constraints during the training phase. The authors tackle this through the ‘Constraint Completion and Correction’ strategy: an *equality completion* that refines partial solutions and an *inequality correction* that amends constraint violations.

Building on the implicit function theorem, DC3 employs a Multilayer Perceptron (MLP) to encode input data x into a partial variable $z \in \mathbb{R}^m$. This is followed by a completion module that ensures adherence to equality constraints. Specifically, using

φ_x , DC3 completes z to $\tilde{y} := \begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix} \in \mathbb{R}^n$. The next step involves *correcting* the variables $\tilde{y}$ to satisfy the inequality constraints by minimizing the loss function ℓ_{ReLU} , where

$$\ell_{\text{ReLU}}(z) = \left\| \text{ReLU} \left(g_x \left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix} \right) \right) \right\|_2^2. \quad (3)$$

The algorithm calculates the derivatives of the $\ell_{\text{ReLU}}(z)$ with respect to z and adjusts the left $n - m$ entries of y to ensure the equality constraints are nearly satisfied. That is, let ρ_x^{DC3} denote one correction step,

$$\rho_x^{\text{DC3}}(\tilde{y}) = \rho_x^{\text{DC3}} \left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix} \right) = \begin{bmatrix} z - \gamma \Delta z \\ \varphi_x(z) - \gamma \Delta \varphi_x(z) \end{bmatrix}, \quad (4)$$

where $\Delta z = \nabla_z \ell_{\text{ReLU}}(z)$, $\Delta \varphi_x(z) = \frac{\partial \varphi_x(z)}{\partial z} \Delta z$.

Here, the learning rate γ is a positive value determined empirically. One can verify that $\rho_x^{\text{DC3}}(\tilde{y})$ satisfies the following equation

$$Jh_x(\tilde{y})(\rho_x^{\text{DC3}}(\tilde{y}) - \tilde{y}) = 0. \quad (5)$$

To ensure that $\tilde{y}$ initially does not violate the constraints too much, the network loss function $f_x(y)$ is replaced with the following penalty function

$$\ell_{\text{soft}}(y) = f_x(y) + \lambda_g \|\text{ReLU}(g_x(y))\|_2^2 + \lambda_h \|h_x(y)\|_2^2, \quad (6)$$

where ReLU is the ReLU function, $\lambda_h \geq 0$ and $\lambda_g \geq 0$ are hyper-parameters.

C. Rethinking of DC3

Correction module: In DC3 method [2], there appears to be a *misconception* that $\rho_x^{\text{DC3}}(\tilde{y})$ satisfies the exact equality constraints. Given the linear approximation $h_x(\tilde{y}) = h_x(\tilde{y}) + Jh_x(\tilde{y})(\tilde{y} - \tilde{y}) + o(\|\tilde{y} - \tilde{y}\|)$ and $h_x(\tilde{y}) \approx 0$, the equality constraints are closely approximated at new point $\rho_x^{\text{DC3}}(\tilde{y})$. Notably, if $h_x(y)$ denotes linear constraints, then $\rho_x^{\text{DC3}}(\tilde{y})$ precisely satisfies the equality constraints. In this paper, we propose a **remedy** to (5), inspired by Newton’s method, which can enhance constraint feasibility.

The second issue of DC3 is that the network architecture does not include any information of the objective $f_x(y)$. Our primary goal is for the network to return a partial variable z that solves the following equivalent problem of (1)

$$\min_z \hat{f}_x(z), \quad \text{s.t.} \quad \hat{g}_x(z) \leq 0, \quad (7)$$

where $\hat{f}_x(z) := f_x\left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix}\right)$, $\hat{g}_x(z) := g_x\left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix}\right)$.

However, DC3 only considers the constraints, making it challenging to reach the optimal solution of (1). Additionally, it can be difficult to simultaneously minimize both the inequality loss and the objective loss by only considering the penalty loss (6). Our experimental findings, illustrated in Fig. 1, provide evidence that minimizing the inequality loss can be particularly challenging in certain cases. It is crucial to recognize that during the correction process, essential information from the objective

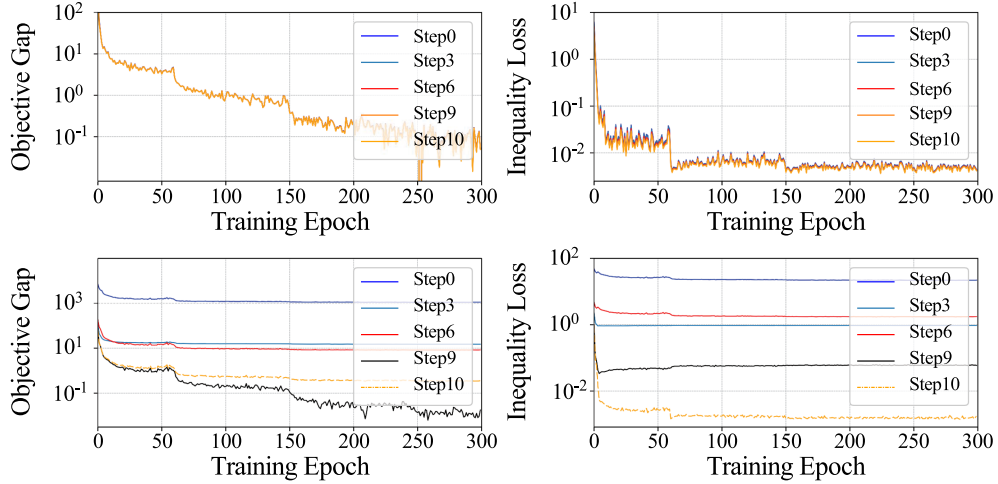


Fig. 1. For the convex quadratic problems in [2], we show the training loss (left) and inequality constraint loss (right) using our learning rate scheduler over 300 epochs. (The original DC3 was trained for 1000 epochs; our scheduler is more efficient.) By selecting outputs $\hat{y}$ at steps 0, 3, 6, 9, and 10 from DC3's correction steps, we measure the objective value and inequality loss. For the 30-inequality-50-equality dataset (top two figures, with 13.5 added to the objective for clear logarithmic scaling), the output at step 0 is nearly feasible. For the 50-inequality-50-equality dataset (bottom two figures, with 13.8 added), the output at step 0 is far from feasible, suggesting more correction steps are needed. Also, the objective does not always decrease as inequality loss is reduced (e.g., steps 9 to 10), indicating possible loss of objective information during correction.

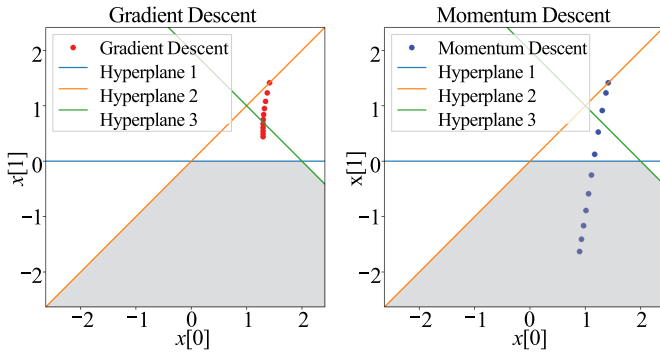


Fig. 2. We use a simple example to illustrate a potential issue during the correction step when applying the ReLU loss function. Three linear inequality constraints ($a_i^\top x \geq b_i$, $i = 1, 2, 3$) are defined. We start from an infeasible point and employ the gradient method and momentum method to minimize $\min_x \sum_{i=1}^3 \|\text{ReLU}(a_i^\top x - b_i)\|^2$. The figure above shows the iteration trajectories. In the case of the gradient descent method, we observe slower convergence rates, necessitating the inclusion of additional layers within the correction modules to achieve active constraints. The momentum method can sometimes enter the interior region, making some of the inequality constraints inactive. This is not good for our optimization problem because some solution lies on the boundary of the feasible region.

function $f_x(y)$ may be lost. Furthermore, in Fig. 2, we demonstrate the use of gradient methods and momentum methods to solve $\min \ell_{\text{ReLU}}(z)$. We observe that a limited number of gradient steps may struggle to attain feasibility, while corrections made using the momentum method can guide the point deeper into the interior of the inequality region. However, this approach may not align with the goal of minimizing $\hat{f}_x$. To address this issue, we will design a new active-set method.

Regularization term: The regularization terms in (6) are essential for maintaining feasibility. However, the minimizer of (6) does not coincide with the minimizer of the original problem (1). In practice, we find that ridge regularization frequently yields a solution with substantial inequality loss and a high objective

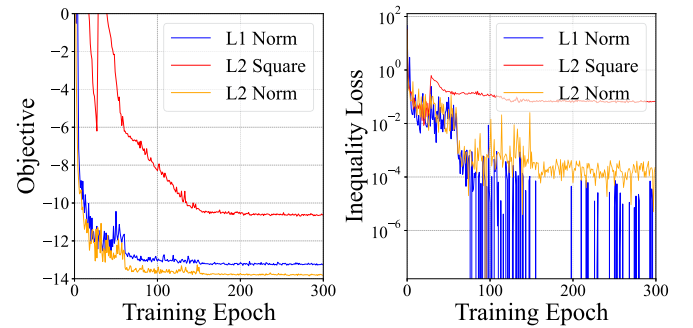


Fig. 3. We test the performance of the L1 norm, the squared L2 norm and the nonsmooth L2 norm. Taking advantage of the sparse property, the L1 norm and nonsmooth L2 norm surpass the squared L2 norm, for both objective and the inequality loss.

value. We also explored L1 regularization and nonsmooth L2 regularization, as shown in Fig. 3. The L2 norm applies a harsher penalty to inequality loss, particularly when there is a significant violation of the constraint. This approach aligns with the exact penalty method in classical optimization theory [3]. Based on these insights, we will propose a new soft loss function that incorporates a nonsmooth L2 norm regularization.

D. Classical Active-Set Method

For convenience, we define the inequality index set $\mathcal{I}$ as

$$\mathcal{I} := \{j \mid \hat{g}_j(z) \leq 0, j = 1, \dots, l\}.$$

Traditionally, for some $j \in \mathcal{I}$, some inequality constraint $\hat{g}_j$ is termed *active* for $\bar{y}$ if $g_j(\bar{y}) = 0$. The collection of active inequality indices, denoted as $\mathcal{A}$, represents the active set. When we identify the active set at a critical point x^* , the inequality-constrained problem can be simplified to an equality-constrained problem. However, identifying this set can be challenging. Therefore, the classical active-set method [3], [4] temporarily

treats the inequality constraints in the active set at each point as equality constraints and then solves the equality-constrained problem to minimize the objective function. It is worth noting that the convergence of the active-set method heavily relies on the precision with which the subproblem is solved.

IV. PROPOSED ALGORITHM

In this section, we propose a new framework to address the limitations of DC3. In Section IV-A, we introduce a new correction module. Compared to DC3, this approach achieves better feasibility for nonlinear constraints. To further reduce the objective value, we introduce a boundary wandering framework in Section IV-B.

A. Modified Correction Step

We first note that after correction step in DC3, the new point $\rho_x^{\text{DC3}}(\tilde{y})$ satisfies (5). However, if $h_x(\tilde{y}) \neq 0$, this is not a good approximation to $h_x(y) = 0$. Hence, we modify the correction step such that it is to guarantee that $\rho_x(\tilde{y})$ satisfies

$$h_x(\tilde{y}) + Jh_x(\tilde{y})(\rho_x(\tilde{y}) - \tilde{y}) = 0, \quad (8)$$

which is equivalent to solve the Newton's linear equation. And we obtain that

$$\rho_x(\tilde{y}) = \rho_x \left(\begin{bmatrix} z \\ \varphi_x(z) \end{bmatrix} \right) = \begin{bmatrix} z - \gamma \Delta z \\ \varphi_x(z) - \gamma \mathcal{H} \varphi_x(z) \end{bmatrix},$$

where $\Delta z = \nabla_z \ell_{\text{ReLU}}(z)$,

$$\mathcal{H} \varphi_x(z) = \frac{\partial \varphi_x(z)}{\partial z} \Delta z + \frac{1}{\gamma} (Jh_2)^{-1} h_x(\tilde{y}). \quad (9)$$

One can also modify Δz to incorporate a momentum step as follows

$$\Delta z = \beta \Delta z + \nabla_z \ell_{\text{ReLU}}(z), \beta \in (0, 1),$$

where β is the momentum parameter. When it comes to nonlinear equality constraints, taking a single step as defined in (9) effectively performs a Newton's step. Consequently, after a sufficiently large number of training steps (typically no more than 10 epochs), we anticipate that the equality constraints will be satisfactorily met.

B. Boundary Wandering Framework

Now, we are ready to introduce an innovative two-stage framework designed to address the other limitation observed in DC3. The first stage leverages the DC3 framework, coupled with our new correction module, to identify a nearly feasible point close to the local optimal point. This is achieved through the application of MLP, completion, and the new correction modules (9). The inspiration for our second stage is drawn from the classical active-set method. In this stage, we execute what we term the "Boundary Wandering Step (BWS)", utilizing the gradient of the objective function to uncover a more optimal solution to problem (1). Our proposed framework, aptly named the Constrained Learning-Based Boundary Wandering Framework (CBWF), is presented in Algorithm 1. To provide further insight

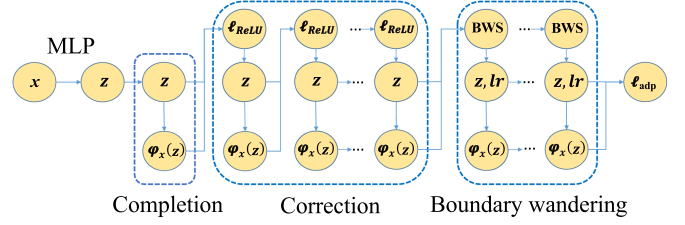


Fig. 4. The diagram displays the workflow of Algorithm 1: CBWF, representing the second stage. In this stage, the MLP is initialized with parameters Θ that were pretrained in the first stage. Next, the partial variable z is completed and corrected to form a feasible solution. The boundary wandering module further reduces the objective value using our BWS method. Finally, we apply the loss function ℓ_{adp} , which will be defined in (22), to train the network using backpropagation.

Algorithm 1: Constraint Learning Based Boundary Wandering framework (CBWF).

- 1: **assume** equality completion procedure
 $\varphi_x : \mathbb{R}^m \rightarrow \mathbb{R}^{n-m}$ {To solve equality constraints}
 - 2: **Procedure:** training process on X
 - 3: **init** neural network $N_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m$, learning rate lr
 - 4: **while** not converged for $x \in X$ **do**
 - 5: **compute** partial variables $z = N_\theta(x)$
 - 6: **complete** the full variables $\tilde{y}$
 $= \begin{bmatrix} z^\top & \varphi_x(z)^\top \end{bmatrix}^\top \in \mathbb{R}^n$
 - 7: **correct** to obtain a feasible solution
 $\hat{y} := \rho_x^{(t_{\text{train}})}(\tilde{y}) \in \mathbb{R}^n$ using (9)
 - 8: **wander** to better solution $\bar{y} := \mathcal{T}(\hat{y}, \text{lr}) \in \mathbb{R}^m$ using Algorithm 2
 - 9: **compute** constraint-regularized loss $\ell_{\text{adp}}(\hat{z})$
 - 10: **update** θ using $\nabla_\theta \ell_{\text{adp}}(\hat{z})$
 - 11: **update** lr using $\nabla_{\text{lr}} \ell_{\text{adp}}(\hat{z})$
 - 12: **end while**
 - 14: **Procedure:** testing process for x using N_θ
 - 15: **compute** partial set of variables $z = N_\theta(x)$
 - 16: **complete** to full set of variables $\tilde{y} = \begin{bmatrix} z^\top & \varphi_x(z)^\top \end{bmatrix}^\top$
 - 17: **correct** to obtain a feasible solution $\hat{y} := \rho^{(t_{\text{test}})}(\tilde{y})$ using (9)
 - 18: **wander** to better solution $\bar{y} := \mathcal{T}(\hat{y}, \text{lr})$ using Algorithm 2
 - 19: **return** $\bar{y}$
-

into its functionality, we illustrate the corresponding data flow in Fig. 4.

In comparison to DC3, our approach introduces an additional wandering step following the correction phase. Specifically, after obtaining $\hat{y}$ through the correction process, we further refine this solution using the wandering step, denoted as $\mathcal{T}(\hat{y}, \text{lr})$, where lr represents the learning rate. What sets our approach apart is the use of a learning-based method to adaptively adjust the step size lr . The details of this wandering strategy are elaborated upon in Algorithm 2, with a comprehensive discussion to follow. Refer to Fig. 5 for a visual depiction of the wandering process.

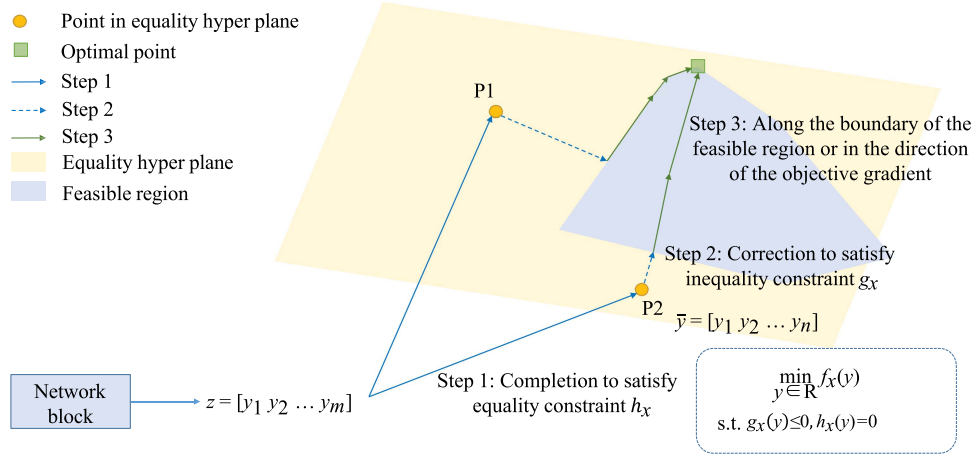


Fig.5. Starting with the output from the neural network, we initially obtain an incomplete solution z . We then complete it to obtain either P_1 or P_2 that satisfies the equality constraints, and apply a correction step to enter the feasible region. The BWS step allows us to navigate the trajectory by either traversing the boundary of the inequalities or directly moving toward the optimal point by following the gradient of the objective function, ultimately guiding us to more favorable solutions.

C. Learning-Based Boundary Wandering Strategy

Our proposed algorithm revolves around two key objectives: (1) the reduction of the objective value and (2) the assurance of feasibility. To accomplish these goals, we have developed a neural network solver tailored for optimization problems. After the correction module, we employ a gradient-based method to further minimize the objective function. In pursuit of this, we introduce a modification to the concept of an active set.

1) *Active Set*: The classical active-set method solves a quadratic subproblem to determine a descent direction, taking into account the active equality constraints specified by the active set. Within our framework, we introduce a degree of tolerance for the active set, with a particular focus on inequality constraints that are in close proximity to binding at the current solution. We introduce a perturbation to the definition of the active set. The number δ_s will be decreasing to 0 during training. Given a small positive number $\delta_s > 0$, for a point z , the active index set $\mathcal{A}$ is defined by

$$\mathcal{A} := \{j \in \mathcal{I} \mid -\delta_s \leq \hat{g}_j(z) \leq 0, j = 1, 2, \dots, l\}.$$

This approach allows for a greater number of inequality constraints to be considered “active.” Consequently, we can obtain improved solutions with the BWS. In scenarios where the active set is not relaxed, BWS may become trapped within a specific inequality constraint region, resulting in minimal progress due to the small learning rate.

Then, BWS allows the optimization trajectory to traverse along the boundary associated with active constraints. Intuitively, by following the negative gradient of the objective function, further reductions in the objective value can be achieved. However, we should carefully control the learning rate lr_s in Algorithm 2 to ensure that the update would not violate the constraint.

Recall that by completion, we solve the equivalent problem (7). To gain a deeper understanding of the problem (7), we examine the KKT conditions at a stationary point z^* . We assume that the linear independent constraint qualification condition

(LICQ) holds for problem (7). Note that LICQ is a standard and mild assumption for constrained problems. Then, the KKT condition of (7) is given by

$$\nabla f_x(z^*) + J\hat{g}_x(z^*)^\top \mu = 0, \quad (10)$$

$$\hat{g}_x(z^*) \leq 0, \quad (11)$$

$$\mu \geq 0, \quad (12)$$

$$\mu_j \hat{g}_j(z^*) = 0, j = 1, \dots, l. \quad (13)$$

By the conditions in (10), (12) and (13), one has $\mu_j = 0, j \notin \mathcal{A}$, and

$$\nabla f_x(z^*) + \sum_{j \in \mathcal{A}} \mu_j \nabla \hat{g}_j(z^*) = 0, \text{ for } \mu_j \geq 0, j \in \mathcal{A}. \quad (14)$$

This implies that $\nabla f_x(z^*)$ should be negatively correlated with some active vectors $\nabla \hat{g}_j(z^*), j \in \mathcal{A}$. On contrary, starting from a point non-optimal point z , we denote its active set as $\mathcal{A}_0$. If $\nabla \hat{g}_j(z)^\top \nabla f_x(z) > 0$ for some $j \in \mathcal{A}_0$, $\hat{f}_x(z)$ can be reduced along $-\nabla f_x(z)$ and guarantee the feasibility for $\hat{g}_j(z) \leq 0$ simultaneously. Given that some inequality constraints might be violated while moving along the gradient descent direction, we introduce an indicator $r_{\mathcal{A}_0} \in \mathbb{R}^{|\mathcal{A}_0|}$ with each element being

$$r_j = \left(\frac{\nabla \hat{g}_j(z)}{\|\nabla \hat{g}_j(z)\|} \right)^\top \nabla f_x(z), j \in \mathcal{A}_0. \quad (15)$$

The indicator $r_{\mathcal{A}_0}$ determines whether the corresponding inequality should be considered. When $r_j < 0, j \in \mathcal{A}$, then walking along $-\nabla f_x(z)$ would violate the constraint $\{z \mid \nabla \hat{g}_j(z) \leq 0\}$. We define the working set $\mathcal{W}_0$ as follows

$$\mathcal{W}_0 := \{j \in \mathcal{A}_0 \mid r_j < 0\}. \quad (16)$$

When handling inequality constraints in the working set $\mathcal{A}_0$, classical active-set method temporarily treat them as equality constraints, which yields the following subproblem

$$\min_d \hat{f}_x(z + d) \text{ s.t. } \nabla \hat{g}_j(z)^\top d = 0, j \in \mathcal{A}_0. \quad (17)$$

Algorithm 2: Boundary Wandering Step (BWS).

```

1: input  $\hat{y}$ ,  $\text{lr}_{\text{init}}$ , objective  $f_x$ , inequality constraints  $g_x$ ,
   completion function  $\varphi_x$ ,  $\delta_s > 0$ ,  $\mathcal{B}_0 = \emptyset$ 
2: initialize  $k = 0$ ,  $z$  from  $\hat{y}$  as in completion, set  $w_0 = z$ 
3: define the objective  $\hat{f}_0(z) = f((z^\top, \varphi_x(z)^\top)^\top)$ 
4: define the inequality constraints
    $\hat{g}_x^0(z) = g_x((z^\top, \varphi_x(z)^\top)^\top) \leq 0$ 
5: calculate the inequality value  $\hat{g}_j(z)$ ,  $j = 1, \dots, l$ 
6: construct the active index set
    $\mathcal{A}_0 = \{j | \hat{g}_j^0(z) \geq -\delta_s, j \in \mathcal{I}\}$  and compute  $r_{\mathcal{A}_0}$  in (15)
7: construct working set  $\mathcal{W}_0$  in (16)
8: while not  $r_{\mathcal{A}_k} \geq 0$  do
9:   update  $k = k + 1$ 
10:  select the most negative  $r_{j_k}$  from  $r_{\mathcal{A}_k}$  with index
      $j_k \in \mathcal{W}_{k-1}$ 
11:  eliminate  $\hat{g}_{j_k}^{k-1}$ 
12:  update  $\mathcal{B}_k = \mathcal{B}_{k-1} \cup \{j_k\}$ ,  $\mathcal{A}_k = \mathcal{A}_{k-1} \setminus \{j_k\}$ 
13:  get  $\varphi_k$  and  $w_k \in \mathbb{R}^{m-k}$  s.t.
      $[w_k^\top, \varphi_k(w_k)^\top]^\top = w_{k-1}$ , and
      $\hat{g}_{j_k}^{k-1}([w_k^\top, \varphi_k(w_k)^\top]^\top) = 0$ ,
14:  define map  $\Phi_k$  in (18) and new functions
      $\hat{f}_k(w_k) = \hat{f}_{k-1}(\Phi_k(w_k))$ ,  $\hat{g}_j^k(w_k) = \hat{g}_j^{k-1}(\Phi_k(w_k))$  for
      $j \in \{1, 2, \dots, l\} \setminus \mathcal{B}_k$ 
15:  compute the gradient  $\nabla_{w_k} \hat{f}_k(w_k)$ 
16:  update  $r_{\mathcal{A}_k} = J_{w_k} \hat{g}_{\mathcal{A}_k}(w_k) \nabla \hat{f}_k(w_k)$ , and working
     set  $\mathcal{W}_k = \{j \in \mathcal{A}_k \mid r_j < 0\}$ 
17: end while
18: let  $K = k$ 
19: let  $\text{lr}_s = \min_j \{\text{lr}_{\text{init}}, \frac{\hat{g}_j^K(w_K)}{\nabla \hat{g}_j^K(w_K)^\top \nabla \hat{f}_K(w_K)}\}$ ,
     where  $j \notin \mathcal{A}_K \cup \mathcal{B}_K$ ,  $\nabla \hat{g}_j^K(w_K)^\top \nabla \hat{f}_K(w_K) < 0$ 
20: update  $w_K = w_K - \text{lr}_s \nabla \hat{f}_K(w_K)$ 
21: recover  $\bar{z} = \Phi_1 \circ \Phi_2 \circ \dots \circ \Phi_K(w_K)$ 
22: return  $\bar{y} = [\bar{z}^\top, \varphi_x(\bar{z})^\top]$ 

```

Directly solving (17) can yield a descent direction, but it may be as computationally expensive as solving the original problem, e.g., use the algorithm in [16]. Furthermore, it is necessary to determine the sign of the dual variable in problem (17) when its solution is $d = 0$. Alternatively, we can employ the completion method, similar to what is done in (9). However, two challenges arise in this approach. First, computing the matrix inverse as shown in (2) during each iteration can be computationally expensive during training. Second, because the temporary active working set may differ from the one at the optimal point, the subproblem (17) may include incorrect active equality constraints, indicated by a negative dual variable. Therefore, we need to carefully address the working set and distinguish the correct active equality constraints from the incorrect ones.

2) *Minimum Selection Rule:* To address these issues, we gradually add one of the binding constraints based on the criterion $r_j < 0$ and adopt a minimum selection rule, meaning we select the most negative r_j . The motivation behind this is that adding one constraint at a time can avoid matrix inversion, and the most negative r_j suggests the most likely

binding inequality at the optimal point. We illustrate this by Example A.1 in Appendix, available online. Algorithm 2 details this process. Specifically, we select the most negative r_{j_1} with index j_1 from $\mathcal{W}_0$ and enforce $\hat{g}_{j_1}(z) = 0$. Subsequently, the variable $z \in \mathbb{R}^m$ can be eliminated to $w_1 \in \mathbb{R}^{m-1}$ and there exists a local map φ_1 such that $z = [w_1, \varphi_1(w_1)^\top]^\top$. And the objective is re-parameterized as $\hat{f}_1(w_1) = \hat{f}_x([w_1, \varphi_1(w_1)^\top]^\top)$. For convenience, we define the operator

$$\Phi_1 : w_1 \rightarrow [w_1^\top, \varphi_1(w_1)^\top]^\top.$$

The inverse of Φ_1 is denoted as $\Phi_1^{-1}(z) = w_1$. Consequently, one has

$$\hat{f}_1(w_1) = \hat{f}_x(\Phi_1(w_1)).$$

Now, one could remove the index j_1 from $\mathcal{A}_0$, and set $\mathcal{A}_1 = \mathcal{A}_0 \setminus \{j_1\}$. We then recompute the indicator $r_{\mathcal{A}_1} = J_{w_1} \hat{g}_{\mathcal{A}_1}(w_1) \nabla \hat{f}_1(w_1) \in \mathbb{R}^{|\mathcal{A}_1|}$. If $r_j < 0$ for $j \in \mathcal{A}_1$, we select j_2 according to minimum selection rule from $\mathcal{A}$ and there exists φ_2 such that $\varphi_2 : \mathbb{R}^{m-2} \rightarrow \mathbb{R}$, and define

$$\Phi_2 : w_2 \rightarrow [w_2^\top, \varphi_2(w_2)^\top]^\top, w_2 \in \mathbb{R}^{m-2}.$$

At iteration k in Algorithm 2, we recursively define the map

$$\Phi_k : w_k \rightarrow [w_k^\top, \phi_k(w_k)^\top]^\top, w_k \in \mathbb{R}^{m-k}, \quad (18)$$

and the objective $\hat{f}_k(w_k) = \hat{f}_{k-1}(\Phi_k(w_k))$ and the constraints $\hat{g}^k(w_k) = \hat{g}^{k-1}(\Phi_k(w_k))$. This process continues iteratively until $r_{\mathcal{A}_K} \geq 0$. In essence, we systematically transform the problem into a smaller-dimensional one by eliminating the inequality constraint one by one according to the minimum selection rule.

Once the $r_{\mathcal{A}}$ is non-negative for all inequalities, we use gradient descent to update w_K . However, one potential issue is that the gradient could be zero due to an incorrectly selected index, such as j_l , as indicated by negative dual variable. Fortunately, the dual variable can be efficiently computed, as demonstrated in Lemma A.8 in Appendix, available online. After discarding the incorrect index j_l , Algorithm 2 can be restarted from j_{l-1} to obtain a new, correct selection path. This process is validated in Lemma A.9 in Appendix, available online. We do not include this procedure in Algorithm 2 since our method is learning-based and only 2 steps of BWS is invoked, which rarely results in encountering zero gradients.

3) *Learning Rate:* Inspired by the active-set method, we set the learning rate to be smaller than both lr_{init} and $\{\frac{\hat{g}_j^K(w_K)}{\nabla \hat{g}_j^K(w)^\top \nabla \hat{f}_K(w)}\}_{j \notin \mathcal{A}_K \cup \mathcal{B}_K}$, where lr_{init} is initialized with $\frac{1}{L}$ and L is the gradient Lipschitz constant. To understand this choice, consider the following approximation

$$\hat{g}_j^K(u) \approx \hat{g}_j^K(w) + \nabla \hat{g}_j^K(w)^\top (u - w). \quad (19)$$

If we have $u - w = -\alpha \nabla \hat{f}_K(w)$, this implies that if $\nabla \hat{g}_j^K(w)^\top \nabla \hat{f}_K(w) < 0$ and $\alpha \leq \frac{\hat{g}_j^K(w)}{\nabla \hat{g}_j^K(w)^\top \nabla \hat{f}_K(w)}$ sufficiently guarantees $\hat{g}_j^K(y) \leq 0$. For nonlinear constraints, the approximation (19) holds locally. Therefore, we propose a safeguard for the learning rate, which enforces $\alpha \leq \frac{\hat{g}_j^K(w)}{\nabla \hat{g}_j^K(w)^\top \nabla \hat{f}_K(w)}$ for all $j \notin \mathcal{A}_K \cup \mathcal{B}_K$, $\nabla \hat{g}_j^K(w)^\top \nabla \hat{f}_K(w) < 0$. Finally, Algorithm

2 returns a solution $\bar{y} = [\bar{z}^\top, \varphi_x(\bar{z})^\top]^\top$ satisfying equality constraints $h_x(\bar{y}) = 0$.

By setting the learning rate as described, we can navigate towards a lower objective value while ensuring that the inequality constraints remain satisfied. Given the costliness of obtaining the local gradient Lipschitz constant, we opt to learn this constant through backpropagation, as detailed in Algorithm 1. This approach allows the algorithm to adaptively approximate a better learning rate based on learned Lipschitz information.

In conclusion, Algorithm 2 solves the following problem utilizing one step of gradient descent of $\hat{f}_K(W_K)$

$$\begin{aligned} \mathcal{P}_K : \quad & \min \quad \hat{f}_x(z), \\ \text{s.t.} \quad & \hat{g}_j(z) = 0, j \in \mathcal{B}_K, \\ & \hat{g}_j(z) \leq 0, j \notin \mathcal{B}_K. \end{aligned}$$

A convergence analysis for the execution time of Algorithm 2 within Algorithm 1 is provided in Theorem A.13 in Appendix A, available online. Although the worst-case execution time of BWS may be similar to that of the simplex method, potentially leading to exponential complexity $O(2^l)$, in practice, the complexity is mainly determined by the number of active sets, which is significantly smaller than the total number of inequality constraints, l . The deep neural network can also provide a better initial feasible point, $\hat{y}$. Our experiments show that typically no more than 5 active sets are found. Let us simply discuss the complexity of Algorithm 2. By LICQ condition, the rank of active inequalities, $|\mathcal{A}_0|$, is smaller than the dimension of z . Therefore, the elimination procedure can be completed within $|\mathcal{A}_0|$ iterations. In practice, since typically only a few inequalities are active, we have observed that no more than five iterations are generally needed for the elimination process to successfully complete in our test problems. The complexity of Algorithm 2 in the worst case is $O(lm|\mathcal{A}_0| + m|\mathcal{A}_0|^2)$. More details can be found in Table VI in the Appendix, available online.

D. Two-Stage Training

In our two-stage framework, we initially train a network to generate a feasible solution $\hat{y}^{DC3}$ using DC3 with correction step (9), then use $\hat{y}^{DC3}$ and the network parameters as input of Algorithm 1 in the second stage. In the first stage, we train the network with the soft loss $\ell_{s_1}(\hat{y})$, defined as

$$\ell_{s_1}(\hat{y}) = f_x(\hat{y}) + \lambda_g \|\text{ReLU}(g_x(\hat{y}))\|_2 + \lambda_h \|h_x(\hat{y})\|_2. \quad (20)$$

In the second stage, we add BWS to achieve a better solution. We define another soft loss $\ell_{s_2}(\bar{y})$ for the output $\bar{y}$ of Algorithm 2

$$\ell_{s_2}(\bar{y}) = f_x(\bar{y}) + \lambda_{g_2} \|\text{ReLU}(g_x(\bar{y}))\|_2. \quad (21)$$

We also guarantee that the input of Algorithm 2, denoted by $\hat{y}$, satisfies feasibility. Therefore, the loss in the second stage is given by

$$\ell_{\text{adp}}(\hat{y}, \bar{y}) = \lambda_w \ell_{s_1}(\hat{y}) + \ell_{s_2}(\bar{y}), \quad (22)$$

where $\lambda_w > 0$ is a parameter.

V. EXPERIMENT

In our experiments, we assess the performance of our method across various optimization problem domains, including convex quadratic programs (QPs), non-convex optimization problems, the challenging AC optimal power flow (ACOPF) problem and a fourth-order objective function. Our focus lies on the objective and constraint loss, with the following key criteria:

- *Optimality*: We gauge the quality of the achieved objective value $f_x(y)$, reflecting how well our method optimizes the objective function.
- *Feasibility*: To measure constraint satisfaction, we consider the maximum violation of constraints, using metrics such as: $\max(\text{ReLU}(g_x(y)))$ for inequality constraints, and $\max(h_x(y))$ for equality constraints.
- *Efficiency*: We evaluate the efficiency of our method in terms of computation time.

In this paper, we evaluate our proposed method¹ against several benchmark approaches:

- *Optimizer*: Traditional optimization solvers, including OSQP [29] and qpth [15] for convex QP settings, and IPOPT [30] for generic non-convex problems.
- *NN*: A straightforward deep learning approach trained to minimize the soft loss defined in (6).
- *DC3*: The complete DC3 framework, incorporating both completion and correction operators, repeated 10 times and trained for 1000 epochs.
- *DC3 (new base)*: The network follows the entire DC3 setting, which includes completion and 5 correction operator 5 times (trained for 300 epochs).
- *DC3+BWS*: DC3 with the addition of two boundary wandering strategy steps after DC3's output, without additional training.
- *Alt-Diff*: [28] is the optimizer for processing convex optimization problems with polyhedral constraints.
- *CBWF*: Our proposed Algorithm 1, designed to effectively optimize both the objective and constraints. We apply the minimum selection rule.
- *CBWF**: Similar to CBWF, but instead of the minimum selection rule, we randomly select the index j_k for which $r_j < 0$.

For all neural networks and the qpth optimizer, we employed full parallelization on a GeForce RTX 2080 Ti GPU, ensuring efficient and concurrent processing.

In the case of the OSQP and IPOPT optimizers, which lack GPU support, we executed them sequentially on an Intel Xeon 2.10 GHz CPU. To simulate full parallelization and provide a fair comparison, we report the total execution time divided by the number of test instances.

We adopt the DC3 framework as our neural network architecture. The DC3's backbone consists of fully connected neural networks with two hidden layers, each containing 200 units, ReLU activation functions, batch normalization [31], and dropout layers applied at each hidden layer. Our neural networks are implemented and trained using the PyTorch.

¹Our code can be accessed at the following URL https://github.com/Clear-fire/bws_optimization.git.

TABLE I
RESULTS ON QP TASK FOR 50-EQUALITY-50-INEQUALITY DATASET

	Obj.value (↓)	Max eq.	Max ineq.	Time(s)
Optimizer (OSQP)	-15.05(0.00)	0.00(0.00)	0.00(0.00)	0.002
Optimizer (qpth)	-15.05(0.00)	0.00(0.00)	0.00(0.00)	1.335
NN	-12.57(0.01)	0.35(0.00)	0.00(0.00)	0.001
DC3	-13.46(0.01)	0.00(0.00)	0.00(0.00)	0.014
DC3 (new base)	-13.79(0.04)	0.00(0.00)	0.00(0.00)	0.008
DC3+BWS	-13.94(0.02)	0.00(0.00)	0.00(0.00)	0.023
Alt-Diff	-14.18(0.00)	0.00(0.00)	0.00(0.00)	1.161
CBWF (Ours)	-14.36(0.05)	0.00(0.00)	0.00(0.00)	0.024
CBWF* (Ours)	-14.17(0.01)	0.00(0.00)	0.00(0.00)	0.022

Our new base DC3 exhibits reduced training and testing times, while CBWF outperforms other deep learning-based methods, achieving the best overall performance in this series of experiments.

TABLE II
RESULTS ON THE SIMPLE NONCONVEX TASK FOR THE
50-EQUALITY-50-INEQUALITY CONSTRAINTS

	Obj.value(↓)	Max eq.	Max ineq.	Time(s)
Optimizer	-11.59(0.00)	0.00(0.00)	0.00(0.00)	0.121
NN	-10.04(0.01)	0.35(0.00)	0.00(0.00)	0.001
DC3	-10.66(0.03)	0.00(0.00)	0.00(0.00)	0.012
DC3 (new base)	-10.90(0.02)	0.00(0.00)	0.00(0.00)	0.006
DC3+BWS	-11.00(0.02)	0.00(0.00)	0.00(0.00)	0.019
CBWF (Ours)	-11.24(0.04)	0.00(0.00)	0.00(0.00)	0.021
CBWF* (Ours)	-11.13(0.02)	0.00(0.00)	0.00(0.00)	0.019

Since this problem is nonconvex, the IPOPT is used as the classical optimizer.

TABLE III
RESULT ON THE HIGH-ORDER DATASET

	Obj.value(↓)	Max eq.(↓)	Max ineq.	Time
Cyipopt	-18.47	1.3e-6	6.8e-7	103
DC3 (new base)	-17.21	<1e-8	<1e-8	0.008
CBWF	-18.01	<1e-8	<1e-8	0.024
CBWF*	-17.97	<1e-8	<1e-8	0.023

TABLE IV
RESULTS ON ACOPT OVER 100 TEST INSTANCES

	Obj.value(↓)	Max eq.	Max ineq.	Time(s)
Optimizer	3.81(0.00)	0.00(0.00)	0.00(0.00)	0.949
NN	3.69(0.02)	0.19(0.01)	0.00(0.00)	0.001
DC3	3.82(0.01)	0.00(0.00)	0.00(0.00)	0.097
DC3 (new base)	3.82(0.00)	0.00(0.00)	0.00(0.00)	0.097
DC3+BWS	3.82(0.00)	0.00(0.00)	0.00(0.00)	0.163
CBWF* (Ours)	3.82(0.00)	0.00(0.00)	0.00(0.00)	0.165

We achieve the same accuracy as the DC3 which has obtained a feasible minimizer (numerical approximation).

For our modified DC3 backbone, we apply a dropout rate of 0.1. The network parameters and learning rate lr are optimized using the Adam optimizer. During training, the initial learning rate is set to 0.01 and is reduced by a factor of 0.1 at epochs 60, 150, and 250, continuing until 300 epochs. This learning rate scheduler enabled effective convergence. In the second stage, the CBWF training, we used an initial learning rate of 0.001, with a decay of 0.1 at epochs 50 and 150, running for a total of 200 epochs. We set the batch size to 128, differing from the original DC3, which used a batch size of 200. For the regularization terms, we assigned specific values: λ_h in (20) was set to 5, and λ_{g_2} in (21) was set to 5.5. Additionally, we used λ_g set to 5 when training the DC3 backbone and 4 when training the BWS. λ_w in (22) was set to 0.5 to balance the regularization effects. The

learning rate of lr_{init} was set to 10^{-10} for efficient training. We applied the BWS strategy twice after the DC3 correction step, resulting in an effective compromise between resource usage and precision. The thresholds δ_s are assigned 0.05 and 0.025 respectively for the two BWS steps.

We conducted multiple runs and averaged the experimental results over 5 repetitions to provide a robust and representative assessment of our method's performance. We followed the partitioning strategy used in DC3, which involved splitting the dataset into training, test, and validation sets with a ratio of 10 : 1 : 1.

A. Comparison With the Competitive Method

Convex problem In the case of convex problems, we ensure consistency with DC3 by utilizing the same training and test datasets. Our evaluation focuses on the following convex optimization problem

$$\begin{aligned} \min_{y \in \mathbb{R}^n} \quad & \frac{1}{2} y^\top Q y + p^\top y, \\ \text{s.t.} \quad & A y = x, \quad G y \leq h, \end{aligned} \quad (23)$$

where the constants are defined as $Q \in \mathbb{R}^{n \times n} \succeq 0, p \in \mathbb{R}^n, A \in \mathbb{R}^{n_{\text{eq}} \times n}, G \in \mathbb{R}^{n_{\text{ineq}} \times n}, h \in \mathbb{R}^{n_{\text{ineq}}}$, and variable $x \in \mathbb{R}^{n_{\text{eq}}}$ varies across different problem instances. In other words, We attempt to approximate the optimal y given x .

In the experiments, to ensure that matrix Q is positive semi-definite, we have set Q to be a diagonal matrix with diagonal elements drawn independently and identically from a uniform distribution on the interval $[0, 1]$. Additionally, the entries of matrices A and G have been randomly generated with independent and identically distributed values from the standard normal distribution. For each problem instance, all entries of the variable x are restricted to the interval $[-1, 1]$. To guarantee the existence of a feasible solution, we have set the inequality values h as follows $h = \sum_j |(GA^+)_{ij}|$, where A^+ is the Moore-Penrose pseudoinverse of A . Therefore, we will get a feasible point $y = A^+ x$, which satisfies $AA^+ x = x$ and $GA^+ x \leq \sum_j |(GA^+)_{ij}|$.

We have used a dataset consisting of 100 variables, 50 inequality constraints, and 50 equality constraints for the experiments in this subsection. We have conducted a comprehensive performance comparison between CBWF and other algorithms, considering objective value, as well as the maximum values of equality and inequality constraint violations, averaged across five experiments. Additionally, we have compared the total runtime on all 833 test instances, assuming full parallelization. We have maintained the original hyperparameters of DC3 when training the network and applying its own correction operators. If any hyperparameters were modified for further analysis, details of these changes are provided separately. Furthermore, we have conducted experiments to assess the impact of varying the numbers of equality and inequality constraints, and the details of these experiments can be found in the Appendix B, available online.

In Table I, although CBWF requires 1.5 times more training time than DC3, it provides a significant advantage in terms of the

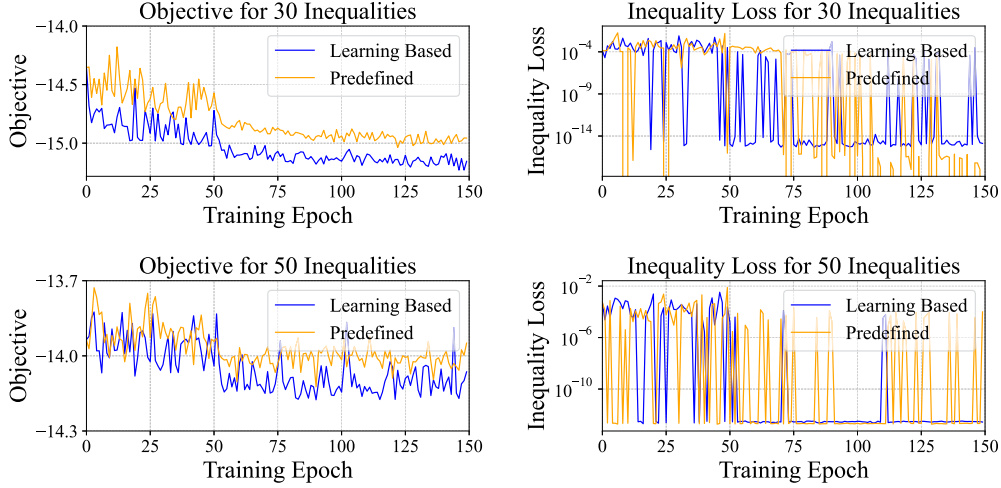


Fig. 6. We evaluated our learning-based strategy using two datasets for the CBWF* algorithm. In both cases, we recorded the objective value and inequality loss after 150 training epochs. The results showed that the learning-based method effectively achieves a better objective value.

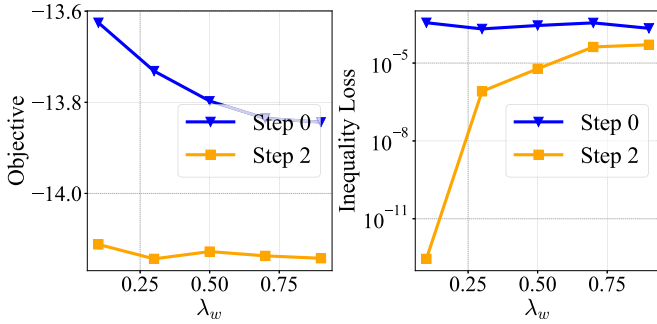


Fig. 7. We observed how the objective and the inequality loss vary with the hyperparameter λ_w in the BWS step 0 and the BWS step 2. λ_w primarily influences the objective for the input and the inequality loss for the output. The choice of λ_w should be made based on the specific problem and the desired balance between the objective and inequality constraints.

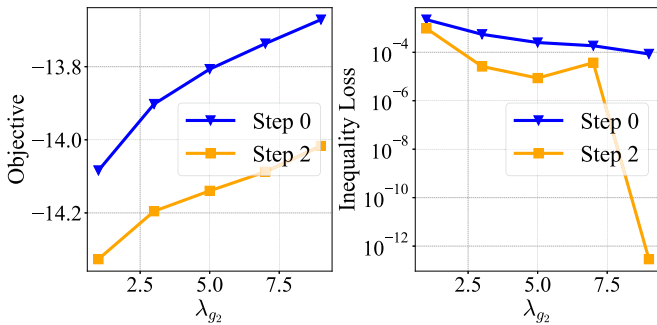


Fig. 8. We analyzed how the objective and the inequality loss vary with the hyperparameter λ_{g_2} in the BWS step 0 and the BWS step 2. λ_{g_2} plays a crucial role in balancing the inequality loss and the objective value. A larger value of λ_{g_2} can degrade the performance of CBWF*, while a smaller value can result in an unacceptable inequality loss. The optimal value of λ_{g_2} depends on the specific problem and trade-off considerations.

objective value. Alt-Diff achieves comparable objective performance, but in each case, its constraint loss is significantly higher than ours. Additionally, CBWF demonstrates better results in terms of inequality loss when compared to the new baseline of DC3 in practical applications. CBWF presents the best objective in the majority datasets, the expectation of inequality loss may

become greater compared to CBWF*, but in every case, we almost find at least one experiment with no greater inequality loss. Additional details and experimental results are provided in Fig. 9. Compared to DC3+BWS, CBWF delivers superior objective value performance and does so without the need for at least 30 BWS iterations to achieve comparable accuracy.

Non-convex problem: In the non-convex setting, we introduce a simple non-convex adaptation of the quadratic program, which includes a sine function in the objective, as follows

$$\begin{aligned} \min_{y \in \mathbb{R}^n} \quad & \frac{1}{2} y^\top Q y + p^\top \sin(y), \\ \text{s.t.} \quad & A y = x, \quad G y \leq h, \end{aligned} \quad (24)$$

where $\sin(y)$ represents the element-wise application of the sine function to each component of the vector y . All parameters of the instances for this problem are drawn randomly as in the convex setting. CBWF outperforms DC3 by 0.47 in terms of the objective value, and it has a 0.13 advantage over DC3+BWS. CBWF achieved a 0.11 improvement in performance compared to CBWF*. For detailed results, please refer to Table II.

AC optimal power flow: AC optimal power flow problem (ACOPF) is a critical problem in the operation of electrical grids. It involves determining the optimal generation of real and reactive power at various nodes of the grid to meet the demand while considering the physics of AC power flow. The problem can be formally defined as follows, where input variables represent real and reactive power demands (p_d and q_d) at different nodes, and the output variables represent real and reactive power generation (p_g and q_g) as well as voltage (v) at those nodes

$$\begin{aligned} \min_{p_g \in \mathbb{R}^b, q_g \in \mathbb{R}^b, v \in \mathbb{C}^b} \quad & p_g^\top A p_g + b^\top p_g, \\ \text{s.t.} \quad & p_g^{\min} \leq p_g \leq p_g^{\max}, \\ & q_g^{\min} \leq q_g \leq q_g^{\max}, \\ & v^{\min} \leq |v| \leq v^{\max}, \\ & (p_g - p_d) + (q_g - q_d)i = \text{diag}(v) \overline{W} \overline{v}. \end{aligned} \quad (25)$$

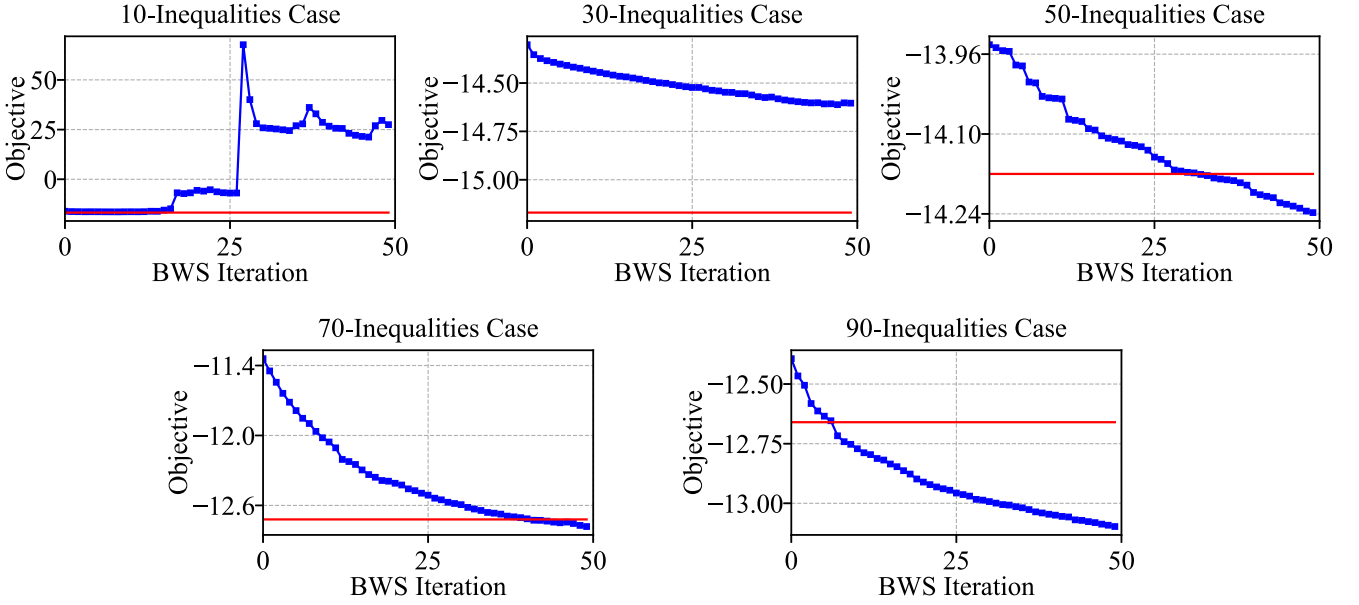


Fig. 9. The comparison between CBWF and DC3+BWS focuses on the number of boundary wandering iterations. In the figure, the red line represents the CBWF results, while the blue points indicate the DC3+BWS results. CBWF requires fewer iterations than DC3+BWS to achieve the same level of optimization accuracy.

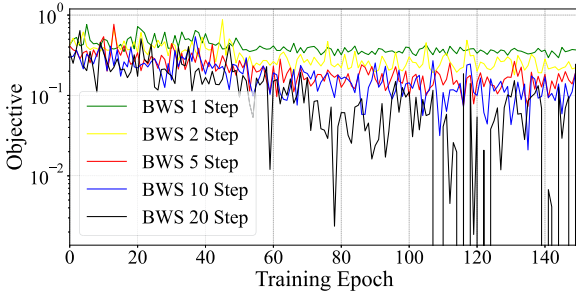


Fig. 10. We performed the experiment on a dataset with 50 equality and 50 inequality constraints for convex programs, adding 14.35 to the objective at each step to facilitate clearer visualization on a logarithmic scale. We tested 1, 2, 5, 10, and 20 steps. We found that as the number of BWS steps increases, the objective value improves.

The objective is to minimize the cost of power generation, considering various constraints such as power balance, voltage limits, and generation limits. In Table IV, we report the results on the same datasets in [2]. This problem has nonlinear equality constraint, thus our CBWF method returns solutions with smaller equality constraint loss, as reported in later ablation study. For the AC optimal power dataset, our approach has achieved comparable performance to the DC3.

Higher-order objective: We have introduced a new high-order synthetic problem as follows

$$\begin{aligned} \min_{y \in \mathbb{R}^n} \quad & \sum_{i=1}^n q_i y_i^4 + p^\top y, \\ \text{s.t.} \quad & Ay = x, \quad Gy \leq h. \end{aligned} \quad (26)$$

We use the same data generation method as in the QP problem to guarantee the existence of a feasible solution. Additionally, we set $q_i > 0, i = 1, \dots, n$ to ensure the problem is bounded. To maintain consistency with previous settings, we randomly

generate a dataset with 50 equality and 50 inequality constraints. To validate our results, we compared them with cypopt. The results are presented in Table III. We found that our CBWF* not only produces better solutions than DC3 but is also significantly faster than the cypopt solver.

B. Ablation Study

In this section, we demonstrate the effectiveness of the learning-based strategy and examine the influence of different hyperparameters in the loss function. We also perform experiments with varying numbers of BWS iterations to highlight the importance of our two-stage training approach.

Learning-based method: Fig. 6 compares the learning-based policy with predefined Lipschitz constants across two datasets of convex quadratic programs. The results clearly show that the learning-based approach consistently outperforms predefined methods in both objective value and inequality loss, demonstrating the effectiveness of our adaptive learning rate strategy in enhancing optimization performance.

Hyper-parameter analysis: We compared the hyperparameters λ_w and λ_{g_2} , which are crucial to the performance of CBWF. The key findings are summarized below.

λ_w (Fig. 7): We observed that variations in λ_w primarily influenced the objective at step 0 (input) and the inequality loss at step 2 (output) of the boundary wandering strategy. A higher λ_w led to improved objectives in the first phase (step 0), but the final objective in the second phase (step 2) did not change significantly. However, a higher λ_w resulted in greater inequality errors, potentially due to the differing weights of λ_{g_2} and λ_g .

λ_{g_2} (Fig. 8): λ_{g_2} is a critical hyperparameter that affects both step 0 and step 2 of CBWF. It plays a role in balancing the inequality loss and the target value. Larger values of λ_{g_2} were found to degrade the CBWF performance, while smaller values made the inequality penalty unacceptable.

TABLE V
RESULT ON THE ACOPT DATASET

Optimizer	Obj.value($\downarrow$)	Max eq.($\downarrow$)	Max ineq.
DC3-	3.812	4e-7	0
DC3	3.817	4.1e-7	1.3e-3
CBWF	3.815	1.1e-6	2.7e-3
CBWF*-	3.816	2.2e-6	1.7e-3
CBWF*	3.815	<1e-8	1.7e-3

We compare the correction steps (4) and (9). “-” indicates the usage of our (4).

BWS iteration evaluation: The comparison between CBWF and DC3+BWS in terms of the number of boundary wandering iterations is illustrated in Fig. 9. DC3+BWS generally requires more than 30 iterations to reach the same accuracy level, while CBWF accomplishes this more efficiently. In the case of 10 inequalities, the performance of our BWS degrades. This degradation is attributed to the approximation of the Lipschitz constant, which is time-consuming and less accurate.

ACOPF equality constraint: In this section, we assessed the effectiveness of (4) and (9) in reducing the equality loss in the case of nonlinear equations in ACOPF. We use DC3- to indicate the usage of (4) in DC3, and CBWF* for using (9) in CBWF*. The results in Table V reveal the improvement in the equality loss achieved by using (9). For more detailed results, please refer to the Appendix B, available online.

BWS steps evaluation: We conducted the relevant experiment shown in Fig. 10. Initially, we used 2 BWS steps. In the Fig. 10, we varied the number of BWS steps and recorded the objective values for 1, 2, 5, 10, and 20 steps. We observed that as the number of BWS steps increases, the objective value improves. However, with a large number of steps, the objective becomes unstable. Thus, we recommend limiting the number of loops to no more than 5, as the extended computation graph complicates long-step learning.

VI. CONCLUSION

In this paper, we present a two-stage learning-based approach for constrained optimization through the Constrained Learning-Based Boundary Wandering Framework (CBWF). CBWF effectively navigates along the boundaries of active constraints, inspired by the active-set method, significantly reducing the objective value while ensuring feasibility. Another key innovation is the modification of the correction module, which greatly enhances equality constraint feasibility, especially on the ACOPT problem. With a soft loss function incorporating a nonsmooth L2 norm and two-stage training, CBWF outperforms existing deep learning-based solvers. This framework offers a robust solution for complex optimization problems across various domains and opens up new possibilities for more applications.

REFERENCES

- [1] J. Wang and P. Srikantha, “Fast optimal power flow with guarantees via an unsupervised generative model,” *IEEE Trans. Power Syst.*, vol. 38, no. 5, pp. 4593–4604, Sep. 2023.
- [2] P. L. Donti, D. Rolnick, and J. Z. Kolter, “DC3: A learning method for optimization with hard constraints,” 2021, *arXiv:2104.12225*.

- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*. Berlin, Germany: Springer, 1999.
- [4] D. P. Bertsekas, “Projected Newton methods for optimization problems with simple constraints,” *SIAM J. Control Optim.*, vol. 20, pp. 221–246, 1982.
- [5] T. Chen et al., “Learning to optimize: A primer and a benchmark,” *J. Mach. Learn. Res.*, vol. 23, pp. 8562–8620, 2022.
- [6] N. Shlezinger, J. Whang, Y. C. Eldar, and A. G. Dimakis, “Model-based deep learning,” in *Proc. IEEE*, vol. 111, no. 5, pp. 465–499, May 2023.
- [7] Z. Fan et al., “Smart initial basis selection for linear programs,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 9650–9664.
- [8] A. Graves, “Generating sequences with recurrent neural networks,” 2013, *arXiv:1308.0850*.
- [9] M. Andrychowicz et al., “Learning to learn by gradient descent by gradient descent,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 3988–3996.
- [10] Y. Cao, T. Chen, Z. Wang, and Y. Shen, “Learning to optimize in swarms,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 15018–15028.
- [11] X. Chen, J. Liu, Z. Wang, and W. Yin, “Theoretical linear convergence of unfolded ISTA and its practical weights and thresholds,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 9079–9089.
- [12] X. Xie, J. Wu, G. Liu, Z. Zhong, and Z. Lin, “Differentiable linearized ADMM,” in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6902–6911.
- [13] N. Parikh and S. Boyd, “Proximal algorithms,” *Found. Trends Optim.*, vol. 1, pp. 127–239, 2014.
- [14] B. Chen, P. L. Donti, K. Baker, J. Z. Kolter, and M. Bergés, “Enforcing policy feasibility constraints through differentiable projection for energy optimization,” in *Proc. ACM Int. Conf. Future Energy Syst.*, 2021, pp. 199–210.
- [15] B. Amos and J. Z. Kolter, “OptNet: Differentiable optimization as a layer in neural networks,” in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 136–145.
- [16] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable convex optimization layers,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 9562–9574.
- [17] J. Bolte, T. Le, E. Pauwels, and T. Silveti-Falls, “Nonsmooth implicit differentiation for machine-learning and optimization,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 13537–13549.
- [18] R. Wang, Y. Zhang, Z. Guo, T. Chen, X. Yang, and J. Yan, “LinSATNet: The positive linear satisfiability neural networks,” in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 36605–36625.
- [19] Z. Gao, Y. Wu, X. Fan, M. Harandi, and Y. Jia, “Learning to optimize on Riemannian manifolds,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 5, pp. 5935–5952, May 2023.
- [20] Z. Gao, Y. Wu, Y. Jia, and M. Harandi, “Learning to optimize on SPD manifolds,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 7697–7706.
- [21] M. Vlastelica, A. Paulus, V. Musil, G. Martius, and M. Rolínek, “Differentiation of blackbox combinatorial solvers,” 2019, *arXiv:1912.02175*.
- [22] Q. Berthet, M. Blondel, O. Teboul, M. Cuturi, J.-P. Vert, and F. Bach, “Learning with differentiable perturbed optimizers,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 9508–9519.
- [23] R. T. Chen, Y. Rubanova, J. Bettencourt, and D. K. Duvenaud, “Neural ordinary differential equations,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 6572–6583.
- [24] E. Dupont, A. Doucet, and Y. W. Teh, “Augmented neural ODE,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 3140–3150.
- [25] S. Bai, J. Z. Kolter, and V. Koltun, “Deep equilibrium models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 690–701.
- [26] S. Bai, V. Koltun, and J. Z. Kolter, “Multiscale deep equilibrium models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 5238–5250.
- [27] A. Pokle, Z. Geng, and J. Z. Kolter, “Deep equilibrium approaches to diffusion models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 37975–37990.
- [28] H. Sun, Y. Shi, J. Wang, H. D. Tuan, H. V. Poor, and D. Tao, “Alternating differentiation for optimization layers,” 2022, *arXiv:2210.01802*.
- [29] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, “OSQP: An operator splitting solver for quadratic programs,” *Math. Program. Comput.*, vol. 12, pp. 637–672, 2020.
- [30] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Math. Program.*, vol. 106, pp. 25–57, 2006.
- [31] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How does batch normalization help optimization?,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 2488–2498.
- [32] A. Achille and S. Soatto, “Information dropout: Learning optimal representations through noisy computation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 40, no. 12, pp. 2897–2905, Dec. 2018.

- [33] M. De Lange et al., “A continual learning survey: Defying forgetting in classification tasks,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 7, pp. 3366–3385, Jul. 2022.
- [34] K. Han et al., “A survey on vision transformer,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 1, pp. 87–110, Jan. 2023.
- [35] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [36] S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
- [37] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” 2016, *arXiv:1607.06450*.



Shuang Wu received the MS degree from the School of Computer Science, Wuhan University, China, in 2019. He is currently working toward the PhD degree with the School of Computer Science, Wuhan University, China. His research interests include deep learning, computer vision and optimization problems.



Shixiang Chen received the PhD degree in systems engineering and engineering management from The Chinese University of Hong Kong, in 2019. He is an assistant professor with the School of Mathematical Sciences, University of Science and Technology of China. He was a postdoctoral associate with the Department of Industrial & Systems Engineering, Texas A&M University. His current research interests include design and analysis of optimization algorithms, and their applications in machine learning and signal processing.



Li Shen received the bachelor's and PhD degrees from the School of Mathematics, South China University of Technology. He is currently an associate professor with Sun Yat-sen University. Previously, he was a research scientist with JD Explore Academy, Beijing, and a senior researcher with Tencent AI Lab, Shenzhen. His research interests include theory and algorithms for nonsmooth convex and nonconvex optimization, and their applications in trustworthy artificial intelligence, deep learning, and reinforcement learning. He has published more than 100 papers in

peer-reviewed top-tier journal papers (*Journal of Machine Learning Research*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *International Journal of Computer Vision*, *IEEE Transactions on Signal Processing*, *IEEE Transactions on Image Processing*, *IEEE Transactions on Knowledge and Data Engineering*, etc.) and conference papers (ICML, NeurIPS, ICLR, CVPR, ICCV, etc.). He has also served as the senior program committee for AAAI and area chairs for ICML, ICLR, ACML, and ICPR.



Lefei Zhang (Senior Member, IEEE) received the BS and PhD degrees from Wuhan University, Wuhan, China, in 2008 and 2013, respectively. He was a Big Data Institute Visitor with the Department of Statistical Science, University College London, U.K., and a Hong Kong Scholar with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China. He is a professor with the School of Computer Science, Wuhan University, Wuhan, China, and also with the Hubei LuoJia Laboratory, Wuhan, China. His research interests include pattern recognition, image processing, and remote sensing. He serves as an associate editor of *IEEE Transactions on Geoscience and Remote Sensing*, an associate editor of *Pattern Recognition*, and a section editor-in-chief of *Remote Sensing*.



Dacheng Tao (Fellow, IEEE) is currently a Distinguished University professor with the College of Computing & Data Science, Nanyang Technological University. He mainly applies statistics and mathematics to artificial intelligence and data science, and his research is detailed in one monograph and more than 200 publications in prestigious journals and proceedings at leading conferences, with best paper awards, best student paper awards, and test-of-time awards. His publications have been cited more than 112 K times and he has an H-index more than 160 in Google Scholar. He received the 2015 and 2020 Australian Eureka Prize, the 2018 IEEE ICDM Research Contributions Award, the 2019 Diploma of The Polish Neural Network Society, and the 2021 IEEE Computer Society McCluskey Technical Achievement Award. He is a fellow of the Australian Academy of Science, AAAS, and ACM.